Airflow 02 Operators

Source originale : https://airflow.apache.org/docs/apache-airflow/stable/core-concepts/operators.html

↑↓

Navigate

■

Select

Esc

Close

Operators

An Operator is conceptually a template for a predefined

Task

, that you can just define declaratively inside your Dag:

with

DAG

(

"my-dag"

)

as

dag

:

ping

=

HttpOperator

(

endpoint

=

"http://example.com/update/"

)

email

=

EmailOperator

(

to

=

"admin@example.com"

,

subject

=

"Update complete"

)

ping

>>

email

Airflow has a very extensive set of operators available, with some built-in to the core or pre-installed providers. Some popular operators from core include:

BashOperator

- executes a bash command

PythonOperator

- calls an arbitrary Python function

Use the

@task

decorator to execute an arbitrary Python function. It doesn’t support rendering jinja templates passed as arguments.

Note

The

@task

decorator is recommended over the classic

PythonOperator

to execute Python callables with no template rendering in its arguments.

For a list of all core operators, see:

Core Operators and Hooks Reference

.

If the operator you need isn’t installed with Airflow by default, you can probably find it as part of our huge set of community

providers

. Some popular operators from here include:

EmailOperator

HttpOperator

SQLExecuteQueryOperator

DockerOperator

HiveOperator

S3FileTransformOperator

PrestoToMySqlOperator

SlackAPIOperator

But there are many, many more - you can see the full list of all community-managed operators, hooks, sensors

and transfers in our

providers packages

documentation.

Note

Inside Airflow’s code, we often mix the concepts of

Tasks

and Operators, and they are mostly

interchangeable. However, when we talk about a

Task

, we mean the generic “unit of execution” of a

Dag; when we talk about an

Operator

, we mean a reusable, pre-made Task template whose logic

is all done for you and that just needs some arguments.

Jinja Templating

Airflow leverages the power of

Jinja Templating

and this can be a powerful tool to use in combination with

macros

.

For example, say you want to pass the start of the data interval as an environment variable to a Bash script using the

BashOperator

:

The start of the data interval as YYYY-MM-DD

date

=

"{{ ds }}"

t

=

BashOperator

(

task_id

=

"test_env"

,

bash_command

=

"/tmp/test.sh "

,

dag

=

dag

,

env

=

{

"DATA_INTERVAL_START"

:

date

},

)

Here,

{{

ds

}}

is a templated variable, and because the

env

parameter of the

BashOperator

is templated with Jinja, the data interval’s start date will be available as an environment variable named

DATA_INTERVAL_START

in your Bash script.

You can also pass in a callable instead when Python is more readable than a Jinja template. The callable must accept two named arguments

context

and

jinja_env

:

The

context

parameter is an Airflow’s

Context

object that provides runtime information for the current task execution. Its contents can be accessed with Python’s standard

dict syntax

. It includes all variables available in Jinja templates and is read-only from the perspective of template rendering - while you can access and use its values, modifications won’t affect the task execution environment.

For a complete list of available context variables see

Templates reference

.

from

typing

import

TYPE_CHECKING

if

TYPE_CHECKING

:

import

jinja2

from

airflow.sdk

import

Context

def

build_complex_command

(

context

:

Context

,

jinja_env

:

jinja2
.
Environment
)
->
str
:
Access runtime information from the context dictionary
task_id
=
context
[
"ti"
]
.
task_id
execution_date
=
context
[
"ds"
]
with
open
(
"file.csv"
)
as
f
:
return
do_complex_things
(
f
,
task_id

,
execution_date
)
t
=
BashOperator
(
task_id
=
"complex_templated_echo"
,
bash_command
=
build_complex_command
,
dag
=
dag
,
)

Since each template field is only rendered once, the callable’s return value will not go through rendering again. Therefore, the callable must manually render any templates. This can be done by calling

render_template()

on the current task like this:

def
build_complex_command
(
context
:
Context
,
jinja_env
:
jinja2
.
Environment

)

->

str

:

with

open

(

"file.csv"

)

as

f

:

data

=

do_complex_things

(

f

)

return

context

[

"task"

]

.

render_template

(

data

,

context

,

jinja_env

)

You can use templating with every parameter that is marked as “templated” in the documentation. Template substitution occurs just before the

pre_execute

function of your operator is called.

You can also use templating with nested fields, as long as these nested fields are marked as templated in the structure they belong to: fields registered in

template_fields

property will be submitted to template substitution, like the

path

field in the example below:

class

MyDataReader

:

template_fields

:

Sequence

[

str

]

=

(

"path"

,)

def

__init__

(

self

,

my_path

):

self

.

path

=

my_path

[additional code here...]

t

=

PythonOperator

(

task_id

=

"transform_data"

,

python_callable

=

transform_data

,

op_args

=

[

MyDataReader

(

"/tmp/{{ ds }}/my_file"

)],

dag

=

dag

,

)

Note

The

template_fields

property is a class variable and guaranteed to be of a

Sequence[str]

type (i.e. a list or tuple of strings).

Deep nested fields can also be substituted, as long as all intermediate fields are marked as template fields:

class

MyDataTransformer

:

template_fields

:

Sequence

[

str

]
=
(
"reader"
,)
def
__init__
(
self
,
my_reader
):
self
.
reader
=
my_reader
[additional code here...]
class
MyDataReader
:
template_fields
:
Sequence
[
str
]
=
(
"path"
,)
def
__init__
(
self
,

my_path

):

self

.

path

=

my_path

[additional code here...]

t

=

PythonOperator

(

task_id

=

"transform_data"

,

python_callable

=

transform_data

,

op_args

=

[

MyDataTransformer

(

MyDataReader

(

"/tmp/{{ ds }}/my_file"

))],

dag

=

dag

,

)

You can pass custom options to the Jinja

Environment

when creating your Dag. One common usage is to avoid Jinja from dropping a trailing newline from a template string:

my_dag

=

DAG

(

dag_id

=

"my-dag"

,

jinja_environment_kwargs

=

{

"keep_trailing_newline"

:

True

,

some other jinja2 Environment options here

},

)

See the

Jinja documentation

to find all available options.

Some operators will also consider strings ending in specific suffixes (defined in

template_ext

) to be references to files when rendering fields. This can be useful for loading scripts or queries directly from files rather than including them into Dag code.

For example, consider a BashOperator which runs a multi-line bash script, this will load the file at

script.sh

and use its contents as the value for

bash_command

:

run_script

=

BashOperator

(

task_id

=

"run_script"

,

bash_command

=

"script.sh"

,

)

By default, paths provided in this way should be provided relative to the Dag's folder (as this is the default Jinja template search path), but additional paths can be added by setting the

template_searchpath

arg on the Dag.

In some cases, you may want to exclude a string from templating and use it directly. Consider the following task:

print_script

=

BashOperator

(

task_id

=

"print_script"

,

bash_command

=

"cat script.sh"

,

)

This will fail with

TemplateNotFound:

cat

script.sh

since Airflow would treat the string as a path to a file, not a command.

We can prevent Airflow from treating this value as a reference to a file by wrapping it in

literal()

.

This approach disables the rendering of both macros and files and can be applied to selected nested fields while retaining the default templating rules for the remainder of the content.

```
from
airflow.sdk
import
literal
fixed_print_script
=
BashOperator
(
task_id
=
"fixed_print_script"
,
bash_command
=
literal
(
"cat script.sh"
),
)
```

Added in version 2.8:

literal()

was added.

Alternatively, if you want to prevent Airflow from treating a value as a reference to a file, you can override

template_ext

:

```
fixed_print_script
=
BashOperator
(
task_id
=
"fixed_print_script"
,
bash_command
```

=

"cat script.sh"

,

)

fixed_print_script

.

template_ext

=

()

Rendering Fields as Native Python Objects

By default, all Jinja templates in

template_fields

are rendered as strings. This however is not always desired. For example, let’s say an

extract

task pushes a dictionary

{"1001":

301.27,

"1002":

433.21,

"1003":

502.22}

to

XCom

:

@task

(

task_id

=

"extract"

)

def

extract

():

data_string

=

'{"1001": 301.27, "1002": 433.21, "1003": 502.22}'

return
json
.
loads
(
data_string
)
If a task depends on
extract
,
order_data
argument is passed a string
"{'1001':
301.27,
'1002':
433.21,
'1003':
502.22}"
:
def
transform
(
order_data
):
total_order_value
=
sum
(
order_data
.
values
())
Fails because order_data is a str :(
return
{
"total_order_value"

:

total_order_value

}

transform

=

PythonOperator

(

task_id

=

"transform"

,

op_kwargs

=

{

"order_data"

:

"{{ ti.xcom_pull('extract') }}"

},

python_callable

=

transform

,

)

extract

()

>>

transform

There are two solutions if we want to get the actual dict instead. The first is to use a callable:

def

render_transform_op_kwargs

(

context

,

jinja_env

):

order_data

=
context
[
"ti"
]
.
xcom_pull
(
"extract"
)
return
{
"order_data"
:
order_data
}
transform
=
PythonOperator
(
task_id
=
"transform"
,
op_kwargs
=
render_transform_op_kwargs
,
python_callable
=
transform
,
)
Alternatively, Jinja can also be instructed to render a native Python object. This is done by passing
render_template_as_native_obj=True
to the Dag. This makes Airflow use

NativeEnvironment
instead of the default
SandboxedEnvironment
:
with
DAG
(
dag_id
=
"example_template_as_python_object"
,
schedule
=
None
,
start_date
=
pendulum
.
datetime
(
2021
,
1
,
1
,
tz
=
"UTC"
),
catchup
=
False
,
render_template_as_native_obj

=

True

,

):

transform

=

PythonOperator

(

task_id

=

"transform"

,

op_kwargs

=

{

"order_data"

:

"{{ ti.xcom_pull('extract') }}"

},

python_callable

=

transform

,

)

Note

NativeEnvironment

renders values according to Python literal rules. This is useful when a template

should produce a list, dict, number, or boolean, but it also means a string that looks like a number,

such as

"42"

, can be rendered as the integer

42

. Keep the default string rendering, use a

callable template field, or add explicit quoting if the task needs the value to stay a string.

Reserved params keyword

In Apache Airflow 2.2.0

params

variable is used during Dag serialization. Please do not use that name in third party operators.

If you upgrade your environment and get the following error:

AttributeError

:

'str'

object

has

attribute

'__module__'

change name from

params

in your operators.

Templating Conflicts with f-strings

When constructing strings for templated fields (like

bash_command

in

BashOperator

) using Python f-strings, be mindful of the interaction between f-string interpolation and Jinja templating syntax. Both use curly braces (

{}

).

Python f-strings interpret double curly braces (

{{

and

}}

) as escape sequences for literal single braces (

{

and

}

). However, Jinja uses double curly braces (

{{

variable

}}

) to denote variables for templating.

If you need to include a Jinja template expression (e.g.,

{{

ds

}}

) literally within a string defined using an f-string, so that Airflow’s Jinja engine can process it later, you must escape the braces for the f-string by doubling them

again

. This means using

four

curly braces:

t1

=

BashOperator

(

task_id

=

"fstring_templating_correct"

,

bash_command

=

f

"echo Data interval start:

{{{{

ds

}}}}

"

,

dag

=

dag

,

)

python_var

=

"echo Data interval start:"

t2

=

BashOperator

(

task_id

=

"fstring_templating_simple"

,

bash_command

=

f

"

{

python_var

}

{{{{

ds

}}}}

"

,

dag

=

dag

,

)

This ensures the f-string processing results in a string containing the literal double braces required by Jinja, which Airflow can then template correctly before execution. Failure to do this is a common issue for beginners and can lead to errors during DAG parsing or unexpected behavior at runtime when the templating does not occur as expected.

Pre- and post-execute methods

The

pre_execute

and

post_execute

methods are called before and after the operator is executed, respectively.

For example, you can use the

pre_execute

method to elegantly determine if a task should be executed or not:

def

_check_skipped

(

context

:

Any

)

->

None

:

"""

Check if a given task instance should be skipped if the `tasks_to_skip` Airflow Variable is a list that contains the task id.

"""

tasks_to_skip

=

Variable

.

get

(

"tasks_to_skip"

,

deserialize_json

=

True

)

if

context

[

"task"

]

.

task_id

in

on_ice

:

raise

AirflowSkipException

(

"Task instance configured to be skipped by `tasks_to_skip` variable."

)

@task

(

pre_execute

=

_check_skipped

)

def

test

():

"""

This task will be skipped if the `tasks_to_skip` Airflow Variable is set and contains the task id.

"""

...

post_execute

can be used to clean up a temporary file or directory that was created by the operator.

The

pre_execute

and

post_execute

methods include the task instance’s context as a parameter.

Difference between pre-/post-execute and setup/teardown

The

pre_execute

and

post_execute

methods are called before and after the operator is executed at the individual task instance level. Setup and teardown are special tasks that are used to before setup or cleanup operations before and after multiple task instances are executed within a Dag run.

Was this entry helpful?